

Etap 2: Proceduralne rozszerzenia w projekcie (3 godz.)

Cel etapu

Zaimplementowanie logiki biznesowej bezpośrednio w silniku bazy danych przy użyciu języka PL/pgSQL. Pozwala to na zapewnienie spójności danych niezależnie od aplikacji klienckiej.

Kiedy używać logiki w bazie danych?

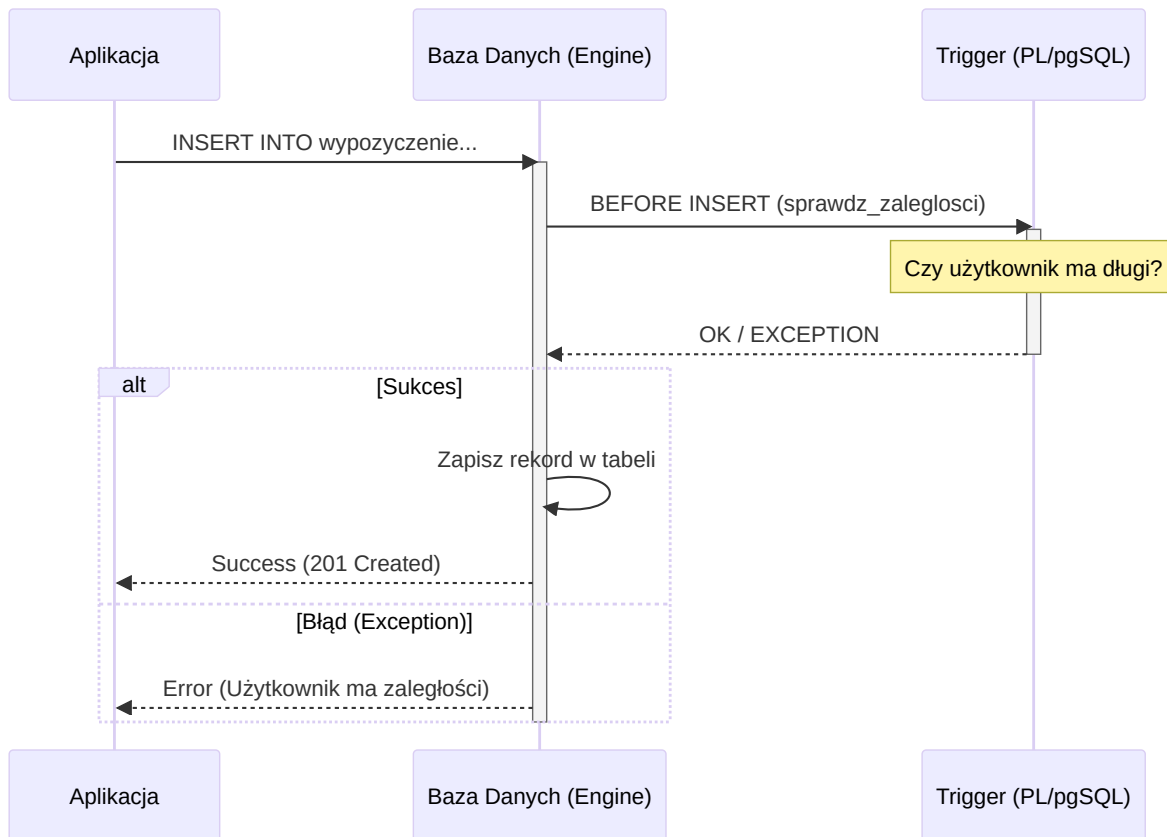
Mechanizm	Kiedy stosować?	Przykład
Wyzwalacz (Trigger)	Automatyczna reakcja na INSERT , UPDATE , DELETE .	Logowanie zmian, walidacja przed zapisem.
Funkcja (UDF)	Powtarzalne obliczenia, transformacje danych.	Obliczanie wieku na podstawie PESEL.
Procedura	Złożone operacje z zarządzaniem transakcjami.	Przeniesienie środków między kontami.

Zadania na tym etapie

Wymagane jest przygotowanie **minimum 3 funkcji** oraz **minimum 3 wyzwalaczy**.

- ☐ **Wyzwalacze (min. 3):**
 - ☐ Wyzwalacz walidacyjny (np. BEFORE INSERT sprawdzający warunki biznesowe).
 - ☐ Wyzwalacz logujący/audytowy (np. AFTER UPDATE zapisujący historię do tabeli pomocniczej).
 - ☐ Wyzwalacz automatyzujący (np. aktualizacja statusu lub daty).
- ☐ **Funkcje (min. 3):**
 - ☐ Funkcja zwracająca tabelę (np. raport aktywności użytkownika).
 - ☐ Funkcja obliczeniowa (np. wyliczanie ceny po rabacie).
 - ☐ Funkcja walidacyjna (np. sprawdzanie poprawności danych wejściowych).

Mechanizm działania wyzwalacza (Trigger)



Przykłady implementacji

1. Wyzwalacz blokujący (PostgreSQL)

Funkcja sprawdzająca status płatności przed nowym wypożyczeniem:

```

-- Funkcja wyzwalająca
CREATE OR REPLACE FUNCTION sprawdz_zaleglosci()
RETURNS TRIGGER AS $$
DECLARE
    ma_zaleglosci BOOLEAN;
BEGIN
    SELECT EXISTS (
        SELECT 1 FROM PLATNOSC p
        WHERE p.uzytownik_id = NEW.uzytownik_id
        AND p.status = 'ZALEGLOSC'
    ) INTO ma_zaleglosci;

    IF ma_zaleglosci THEN
        RAISE EXCEPTION 'Użytkownik posiada zaległości w płatnościach';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
  
```

```
-- Wyzwalacz
CREATE TRIGGER trg_sprawdz_zaleglosci
BEFORE INSERT ON WYPOZYCZENIE
FOR EACH ROW
EXECUTE FUNCTION sprawdz_zaleglosci();
```

2. Funkcja walidująca (Regex)

```
CREATE OR REPLACE FUNCTION waliduj_email(email TEXT)
RETURNS BOOLEAN AS $$
BEGIN
    RETURN email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$';
END;
$$ LANGUAGE plpgsql;
```

Integracja z Pythonem (opcjonalnie)

Jeśli chcesz dodać warstwę aplikacyjną w Pythonie, dla PostgreSQL użyj biblioteki `psycopg` i zarządzaj transakcjami w kodzie aplikacji. Dla prototypów SQLite można użyć `sqlite3` i rejestrować funkcje niestandardowe.

Lista kontrolna (Checklist)

- ☐ Czy zaimplementowałeś **minimum 3 funkcje**?
- ☐ Czy zaimplementowałeś **minimum 3 wyzwalacze**?
- ☐ Czy każda funkcja/wyzwalacz ma krótki opis działania?
- ☐ Czy funkcje mają zdefiniowany język (`LANGUAGE plpgsql`)?
- ☐ Czy wyzwalacze typu `BEFORE` są używane do walidacji, a `AFTER` do logowania (audit)?
- ☐ Czy zdefiniowałeś tabelę pomocniczą dla wyzwalaczy logujących?